Technical Specification

Space Documentation Twin
Challenge 2 Documentation scarcity | Consortium PoA
Date: 26 Feb 2026

1. Scope and targeted challenge

This design targets Challenge 2 documentation scarcity. For many COTS components, radiation and reliability information is missing or locked behind private contracts like NDAs. Small companies' QA teams either take unsatisfied decisions based on the weak signals or repeat the same expensive tests, and results are not shared. The goal here is not to solve everything. We only improve one part: make it easier to publish and discover test documents, with a clear payment flow.

2. System overview

The system is a consortium blockchain network (Proof of Authority). Validators are known organizations (space agency). A batch of physical parts is represented by one ERC-721 token (the digital twin). A test house can publish a report pointer (CID) and set a fee. Buyers can pay-per-access to unlock the pointer.

Main roles:

- Consortium admin: approves minters and test houses; maintains a small validator set.
- Minter (distributor): mints the NFT twin for a batch and sets the initial metadata pointer.
- Test house: publishes the radiation report pointer and receives fees.
- Buyer (QA): buys access to view the report pointer and download the PDF from IPFS.

3. Smart contract specification

Contract file: contracts/SpaceDocTwin.sol. It is an ERC-721 NFT with role-based access control. It stores only minimal data on-chain and uses IPFS/Arweave CIDs for large documents.

3.1 Global variables

Variable	Type	Meaning
tokenCounter	uint256	Incremental NFT id counter.
minStakeWei	uint256	Minimum stake required to publish a report (anti-spam + discipline).
DISPUTE_WINDOW	uint256	Time window before a test house can withdraw its stake.

3.2 On-chain storage layout

Main struct and mappings:

Name	Type	Purpose
TwinData	struct	Minimal batch data + pointers (CIDs) + fee/stake.
registry[tokenId]	mapping(uint256 => TwinData)	Lookup twin data by tokenId.
hasAccess[tokenId][user]	mapping(uint256 => mapping(address => bool))	Tracks who paid for access to reportCID.

3.3 Roles

Role	Who holds it	What it allows
DEFAULT_ADMIN_ROLE	consortium admin	Assign/revoke roles, set parameters.
MINTER_ROLE	distributor	Mint twins and update metadata pointer.
TEST_HOUSE_ROLE	approved lab	Publish reportCID and set fee (with stake).
ARBITER_ROLE	agency	Slash stake in case of proven fraud; view reportCID if needed.

3.4 Events

Events are used for indexing and audit. They allow an off-chain indexer to build a searchable timeline.

- TwinMinted
- MetadataUpdated
- ReportPublished
- AccessPurchased
- DisputeOpened
- StakeSlashed
- StakeWithdrawn

3.5 Special functions

Core functions:

Function	Caller	What it does
mintTwin(to, lotHash, partNumberHash, metaCID)	MINTER_ROLE	Mints a new ERC-721 twin and stores the metadata pointer.
updateMetaCID(tokenId, newMetaCID)	owner or MINTER_ROLE	Updates the metadata pointer when off-chain JSON is extended.

publishReport(tokenId, reportCID, feeWei)	TEST_HOUSE_ROLE	Publishes report CID, sets fee, locks stake.
buyAccess(tokenId)	buyer	Pays exact fee, records access, forwards payment to test house.
getReportCID(tokenId)	owner / paid user / ARBITER	Gated read of report CID.
openDispute(tokenId, evidenceCID)	any	Opens a dispute with off-chain evidence pointer.
slashStake(tokenId, amountWei)	ARBITER_ROLE	Reduces stake if fraud is proven.
withdrawStake(tokenId)	test house	Withdraws stake after dispute window.

4. Architecture diagram and network setup

The system has a simple flow: user app -> RPC gateway -> PoA validators -> on-chain state/events. Large files stay off-chain (IPFS pinning). An indexer listens to events and provides search.

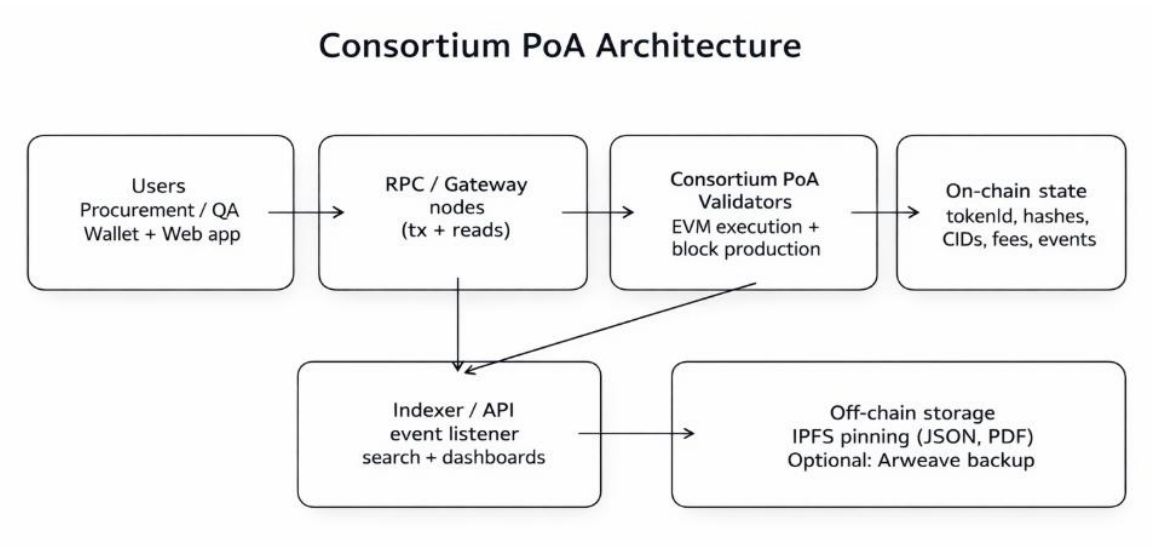

Figure 1. Hardware + network + storage overview (consortium PoA).

4.1 EVM and consensus mechanism considerations

EVM: the smart contract runs on an Ethereum-compatible execution layer. Consensus: Proof of Authority (PoA) in a consortium. We choose PoA because latency matters in operational workflows, and validators should be known and accountable in a regulated industry. This is a trade-off: less decentralised than public PoS, but easier for a pilot and for IP-sensitive data.

4.2 Node roles

The network has different node roles. RPC nodes receive user transactions and relay them. Transactions wait in the mempool. Validators run execution and consensus clients (execution re-

runs the EVM logic; consensus builds blocks and agrees on final state). Indexers read events and build a searchable layer for users.

5. Data storage setup (on-chain vs off-chain)

Data element	Stored where	Reason
tokenId, owner	On-chain	Uniqueness and ownership transfer.
lotHash, partNumberHash	On-chain	Small identifiers; hashes reduce leakage.
metaCID, reportCID	On-chain	Short pointers to large files.
fees, payments, access rights, events	On-chain	Financial truth + audit trail.
JSON metadata	Off-chain (IPFS)	Large and evolves over time.
Radiation report PDF	Off-chain (IPFS)	Too large for on-chain storage; needs long-term availability.

Integrity idea: if a PDF is changed, the CID changes. The chain keeps the expected CID, so tampering is visible.

6. Data models and API calls

We use one main data model (metadata JSON) and one small output model (documentation completeness score). The API flow pulls data from on-chain and off-chain locations.

6.1 Metadata data model (off-chain)

Key fields:

Field	Type	Meaning
part_name	string	Human name of the component.
manufacturer	string	Manufacturer name.
standards	list[string]	Any available indicators (e.g., JEDEC/AEC-Q).
documents.datasheet	string (URI)	Pointer to datasheet.
documents.pcn	string (URI)	Pointer to PCN / change notice.
documents.radiation_report	string (URI)	Pointer to radiation report (same as reportCID content).
radiation_summary.tid_krad	number	Total ionizing dose summary.
radiation_summary.pass	bool	Simple pass/fail label for quick view.

6.2 API calls

The idea is: (1) read metaCID from the contract, then (2) fetch the JSON from IPFS, then (3) compute a score.

```
# 1) on-chain: read metaCID from registry(tokenId)
twin = contract.functions.registry(token_id).call()
metaCID = twin[2]

# 2) off-chain: fetch JSON metadata from IPFS
meta = requests.get(f"https://ipfs.io/ipfs/{metaCID}", timeout=10).json()

# 3) model output: documentation completeness score
result = compute_doc_score(meta)
print(result.score, result.label)
```

Full demo script: fetch_and_score.py

The on-chain NFT stores minimal truth (ownership, pointers, payments). The off-chain JSON is the structured documentation package. The Python script demonstrates how an engineer can pull on-chain pointers and off-chain documents to produce a simple documentation completeness signal, which directly addresses documentation scarcity.

6.3 Model output: Documentation Completeness Score

Output is a simple score from 0 to 100. It helps procurement compare how complete the documentation is. Example weights: radiation report (40), datasheet (20), PCN (20), standards indicator (20). This directly supports Challenge 2, because the main problem is missing evidence.

7. Design motivations and basic scalability notes

Motivation is to create a small market for missing documents. Test houses earn fees. Buyers save time and reduce repeated testing. The pilot should be small (few validators, few labs, few components). If it works, the network can add more participants. More details (tokenomics and scaling path) are in Document 2.